

1. System Overview

Vote4U is a web-based 3D interactive application that simulates the election process using a combination of frontend rendering, 3D visualization, AI-generated explanations, and cloud-based data storage.

The system is designed to be:

- Fast to build (hackathon-ready)
 - Smooth to run (demo-safe)
 - Visually engaging (3D experience)
-

2. Architecture

Core Stack:

- **Frontend:** JavaScript (AI-generated via Google Antigravity)
 - **3D Engine:** Three.js
 - **Backend / Database:** Supabase
 - **AI Integration:** API-based (Gemini / OpenAI)
 - **Version Control:** GitHub
-

Architecture Flow:

User → Frontend UI → 3D Scene (Three.js)
↓
Interaction Handler (JS Logic)
↓
Supabase (store votes)
↓
AI API (generate explanations)
↓
UI Response (display results + info)

3. User Flow (System Level)

1. User enters web app
 2. 3D scene loads (lightweight optimized assets)
 3. User navigates using hybrid controls:
 - Click-based movement
 - UI buttons
 4. User actions:
 - Register → stored temporarily
 - Vote → stored in Supabase
 - View results → fetched and displayed
 5. AI explanation triggered at each step
-

4. Component Breakdown

◆ A. 3D Scene Layer

- Built using Three.js
- Contains:
 - Registration Booth
 - Voting Area
 - Results Zone

👉 Optimized with:

- Low-poly models
 - Minimal textures
 - Lazy loading
-

◆ B. UI Layer

- Overlay interface (HTML/CSS/JS)
- Includes:
 - Input forms
 - Buttons

- Result panels

◆ C. Interaction Engine

Handles:

- Click detection
- Scene transitions
- User actions

◆ D. Database (Supabase)

Stores:

- Votes
- Candidate selection

Structure:

- Table: votes
 - id
 - candidate
 - timestamp

◆ E. AI Engine

API-based integration for:

- Step explanations
- User guidance

Trigger points:

- Registration
- Voting
- Results

5. AI Integration Design

Approach:

- Use API calls only when needed (to save time + cost)
 - Combine:
 - Pre-defined prompts
 - Dynamic user context
-

Example Flow:

User clicks “What is voting?” →

Frontend sends prompt →

AI API →

Returns simplified explanation →

Displayed in UI panel



6. Data Handling Strategy

- Votes stored in Supabase (persistent)
 - Session data handled locally
 - No authentication required (MVP)
-



7. Performance Strategy

To balance **heavy visuals + smooth experience**:

- Use low-poly 3D models
 - Limit real-time rendering load
 - Avoid unnecessary animations
 - Load scenes step-by-step
-



8. Deployment Strategy

- Code hosted on GitHub
- Live deployment via:
 - Vercel (preferred for speed)

9. Constraints & Decisions

Chosen:

- Three.js → for control + visual impact
- Supabase → for simple backend
- API AI → for dynamic explanation

Avoided:

- Complex backend logic
- Multiplayer features
- Heavy asset loading

10. Why This Design Wins

- Matches hackathon theme (prompt-driven + AI)
 - Balanced complexity (not overbuilt)
 - Strong demo flow
 - Scalable idea with minimal implementation risk
-